

University of Vigo

SI – Course 2025–2026

WAREHOUSE PROJECT

İbrahim Emir Tosun-Erasmus Student
S25862950

Submission date: May 2026

1. Introduction

This document describes my work on a multi-agent system that manages an automated logistics warehouse. The system is built on Jason 3.3.0, with the agents written in AgentSpeak and the environment in Java. It was developed for the Intelligent Systems course in the 2025–2026 academic year.

The warehouse handles three container types — urgent, standard, and fragile — and stores them on shelves following a strict policy. When a type runs out of shelf space, an output cycle is triggered: the system stops accepting new containers of that type and the robots start moving the stored ones to the outbound area. Each cycle has its own deadline, and any container that is not delivered in time is reported as a violation, but the system keeps running.

The main things the system has to do:

- Receive containers and store them on the right shelves.
- Use S1, S5 and S8 for urgent containers, and S2, S3, S4, S6, S7, S9 for standard and fragile.
- Detect when a type has no free space and start an output cycle.
- Use deadline $T_0 + \Delta T$ for urgent and $T_0 + 3 \cdot \Delta T$ for non-urgent.
- During a cycle, only move containers of the active type.
- Print structured EVENT logs from supervisor, scheduler and robots.
- Let the robots decide which container to take by themselves.
- Keep the environment thin: it gives state, the agents take the decisions.

2. System Architecture

The system has two layers. On one side there is the Java environment, which simulates the warehouse: the grid, the containers, the shelves and the robot positions. On the other side there are the seven Jason agents that take all the decisions. Agents read percepts published by the environment and act on it through the external actions defined in WarehouseArtifact.

The seven agents and what each of them does:

Agent	Responsibility
robot_light	Carries small light containers (≤ 10 kg, 1×1).
robot_medium	Carries medium containers (≤ 30 kg, 1×2).
robot_heavy_1, robot_heavy_2	Two heavy instances (≤ 100 kg, 2×3).
supervisor	Detects no-space; reports missed deadlines.
scheduler	Computes T_0 and the deadline; opens and closes cycles.
transport	External truck: removes delivered containers from outbound.

Note that the environment never decides whether a shelf is full, when a deadline is missed, or when a cycle should start or close. All those decisions live in the supervisor and the scheduler. That is what keeps the environment thin.

3. The BDI Model in this Project

Jason works with the Belief–Desire–Intention model. Every agent has beliefs (the things it considers true at a given moment), goals it wants to reach, and plans it picks to reach them. Below I show how each of these three pieces shows up in my agents.

3.1 Beliefs

Beliefs come from two sources:

- Initial beliefs in .asl files: e.g. `agent_id(robot_medium)`, `max_weight(30)`, `max_size(1,2)`, `state(idle)`.
- Percepts published by the environment: `container_available(CId,W,H,Weight,Type)`, `shelf_state(Shelf,FreeWeight,FreeVolume)`, `output_pending(CId,Type)`, `cycle_active(Type)`.

The supervisor also keeps `allowed_shelf(Type, Shelf)` facts and a Prolog-like rule, `storage_possible/4`, that it uses to ask itself the question "is there any allowed shelf with enough space for this container?"

3.2 Desires (Goals)

- Robots: `!work_cycle` keeps the robot looking for tasks.
- Supervisor: `!monitor` checks storage and deadlines once per second.
- Scheduler: `!cycle_monitor` closes a cycle when no container of the active type is pending.
- Transport: reacts to the achievement goal `!clear_outbound(Type)`.

3.3 Intentions (Plans)

Plans have the shape `+event : context <- body`. When several plans match the same event, Jason picks the first one whose context is true. Robots have three versions of `!work_cycle`. The most interesting one fires when there is an output candidate and the robot can carry it:

```
+!work_cycle
: state(idle) & output_candidate(CId,W,H,Weight,Type,ShelfId)
  & max_weight(MaxWeight) & max_size(MaxW,MaxH)
  & Weight <= MaxWeight & W <= MaxW & H <= MaxH
<-
  -+state(claiming);
  claim_output(CId);
  .wait(350);
  !work_cycle.
```

There is a similar plan for inbound storage (only when no cycle is active) and a fallback that just waits and retries. So in every iteration the robot looks at its beliefs again and decides what to do next, which is the BDI reasoning loop in practice.

4. Warehouse Layout and Shelf Policy

The warehouse is a 20×15 grid. There are three fixed zones: Inbound at $x \in [0,2]$, $y \in [0,1]$, Classification at $x \in [3,6]$, $y \in [0,1]$ and Outbound at $x \in [17,19]$, $y \in [12,14]$. Nine shelves are placed at three different rows, with three capacity sizes: small (50 kg / 8 vol), medium (100 kg / 12 vol) and large (200 kg / 20 vol).

The shelf policy is enforced in two places:

- In the environment, `drop_at(Shelf)` is rejected when the shelf is not allowed for that container type. This is the hard rule.
- In the supervisor, `allowed_shelf` and `shelf_state` are combined through `storage_possible/4` to decide if a `no_space_detected` event should be raised.

```
allowed_shelf(urgent,"shelf_1"). allowed_shelf(urgent,"shelf_5").
allowed_shelf(urgent,"shelf_8").
allowed_shelf(standard,"shelf_2"). ... allowed_shelf(fragile,"shelf_9").

storage_possible(Type,W,H,Weight) :-
    allowed_shelf(Type,Shelf)
    & shelf_state(Shelf,FreeWeight,FreeVolume)
    & Area = W * H & Weight <= FreeWeight & Area <= FreeVolume.
```

5. Agent Interactions

Agents talk to each other in two ways. The first one is the environment itself: every percept the environment publishes is visible to all agents, and that works as a shared blackboard. The second one is direct messaging with `.send`, used by supervisor, scheduler and transport. Robots never message each other; they only react to what they see.

The full sequence of an output cycle goes like this:

1. Supervisor reads `shelf_state` and `container_available` percepts every second.
2. If a waiting container of some type cannot fit on any allowed shelf, supervisor prints `no_space_detected` and sends `activate_output_cycle(Type)` to scheduler.
3. Scheduler reads the current time as T_0 , computes the deadline ($T_0 + \Delta T$ or $T_0 + 3 \cdot \Delta T$) and calls `open_output_cycle(Type)`.
4. Environment publishes `output_candidate` and `output_pending` percepts for that type, and sets `cycle_active(Type)`.
5. Scheduler prints `output_phase_started` and `deadline_started`, sends `active_deadline(Type, Deadline)` to supervisor and `clear_outbound(Type)` to transport.
6. Robots see the new `output_candidate` percepts, claim the ones they can carry, move to the shelf, pickup, deliver and print `container_delivered`.
7. Every second, supervisor checks active deadlines. If $T > \text{Deadline}$ and a container is still pending, it prints `deadline_missed` once for that container.
8. When no `output_pending(_,Type)` remains, scheduler calls `close_output_cycle(Type)`, prints `deadline_ended` and tells supervisor `cycle_closed(Type)`.

6. Output Cycles and Deadlines

6.1 Starting a Cycle

A cycle starts when the supervisor sees a container waiting at the inbound area whose type does not fit on any allowed shelf any more. The plan context has more conditions than that on purpose:

```
+!check_storage
  : not cycle_active(_)
    & container_available(CId,W,H,Weight,Type)
    & stored_type(Type)
    & not storage_busy(Type)
    & not no_space_reported(Type)
    & not storage_possible(Type,W,H,Weight)
<-
  read_time; ?time(T);
  +no_space_reported(Type);
  .print("EVENT | time=", T, " | agent=supervisor | type=no_space_detected | data=",
Type);
  .send(scheduler, achieve, activate_output_cycle(Type)).
```

Why these extra conditions? `storage_busy(Type)` means another robot is already on its way to a shelf with a reservation, so I should wait before complaining. `stored_type(Type)` means I only open a cycle for a type that already has at least one container on the shelves; otherwise the cycle would have nothing to do. And `no_space_reported(Type)` makes sure the `no_space_detected` event is printed only once per blocked situation, not every second.

6.2 ΔT Value and Deadline Computation

I picked $\Delta T = 30$ seconds. The scheduler keeps it as a belief `delta_t(30)` and computes the deadline depending on the type:

```
+!deadline_for(urgent,T0,Deadline) : delta_t(DT) <- Deadline = T0 + DT.
+!deadline_for(standard,T0,Deadline) : delta_t(DT) <- Deadline = T0 + 3*DT.
+!deadline_for(fragile,T0,Deadline) : delta_t(DT) <- Deadline = T0 + 3*DT.
```

Why 30 s? Robots move at 90, 140 or 220 ms per cell, depending on type, and a full output task is around 30–40 grid steps. In my actual runs the delivery time per container went from 4 to 29 seconds, with most deliveries finishing in 4–10 seconds. So 30 seconds is enough to deliver most urgent containers in a normal situation, but small enough that a missed deadline is possible if the queue gets long. Non-urgent types use $3 \cdot \Delta T = 90$ s, which is the 3:1 ratio asked for in the assignment.

6.3 Closing a Cycle and the `cycle_ready` Flag

An earlier version of my scheduler had a bug: it was closing the cycle right after opening it. The reason was a small race condition. Between calling `open_output_cycle` and the moment the first `output_pending` percepts arrived back at the scheduler, the closing context `not output_pending(_,Type)` was already true, so the cycle ended before any robot could even start. I fixed it with a small confirmation step:

```
+!activate_output_cycle(Type) : not cycle_active(_) <-
  read_time; ?time(T0);
  !deadline_for(Type,T0,Deadline);
  open_output_cycle(Type);
  .wait(500);
```

```

!confirm_cycle(Type,T0,Deadline).

+!confirm_cycle(Type,T0,Deadline) : cycle_active(Type) <-
+active_cycle(Type,Deadline);
.print(... output_phase_started ...);
.print(... deadline_started ...);
.send(supervisor, tell, active_deadline(Type,Deadline));
.send(transport, achieve, clear_outbound(Type));
.wait(1500);
+cycle_ready(Type).

```

Now the scheduler only thinks about closing the cycle when `cycle_ready(Type)` is true, which removes the race entirely. Closing the cycle is then straightforward:

```

+!cycle_monitor
: active_cycle(Type,Deadline) & cycle_ready(Type)
& not output_pending(_,Type)
<-
read_time; ?time(T);
close_output_cycle(Type);
-active_cycle(Type,Deadline); -cycle_ready(Type);
.print("EVENT | time=", T, " | agent=scheduler | type=deadline_ended | data=", Type);
.send(supervisor, tell, cycle_closed(Type));
.wait(1000); !cycle_monitor.

```

6.4 Reporting Missed Deadlines

The supervisor checks active deadlines once per second. If a container is still pending and the current time has already passed the deadline, it prints one `deadline_missed` event for that container. The `missed(CId)` belief makes sure each container is reported at most once. These violations are informative — the system does not stop, robots keep delivering whatever is left, and the cycle still closes when the queue is finally empty.

7. Robot Autonomy: Why a Claim-Based Design

One of the trickier design choices was how the robots should decide what to do. I had two options: a scheduler that picks a robot for each task and sends a message, or a claim-based design where the robots watch percepts and try to grab a task themselves.

I went with the claim-based design for a few reasons:

- It fits the autonomy described in the project description — the scheduler is not supposed to give individual orders to the robots.
- It keeps the scheduler simple. It does not need to know about robot capacities, positions or load. Each robot just looks at its own beliefs.
- Adding a new robot type is just adding another .asl file. The scheduler does not change.

The race between robots is handled by the environment: claim_storage and claim_output are both synchronized inside Java, so only the first robot wins. The others just get a claim_failed percept and silently go back to !work_cycle to try something else.

8. Logging and Traceability

All required events follow the format below:

EVENT time=T agent=NAME type=EVENT_TYPE data=VALUE		
Agent	Event type	When
supervisor	no_space_detected	When a waiting container of a type has no allowed shelf with enough space.
supervisor	deadline_missed	Once per pending container after its deadline has passed.
scheduler	output_phase_started	When an output cycle is opened (T0).
scheduler	deadline_started	Same instant as output_phase_started.
scheduler	deadline_ended	When the cycle closes.
robot_*	container_delivered	When a robot drops a container in the outbound area.

The trace below was captured from a real run with gradle run. It shows one complete standard output cycle that started at T0=82 s when the standard shelves filled up, and closed at 111 s, 29 seconds later — well inside the 90 s non-urgent deadline:

EVENT	time=82	agent=supervisor	type=no_space_detected	data=standard
EVENT	time=82	agent=scheduler	type=output_phase_started	data=standard
EVENT	time=82	agent=scheduler	type=deadline_started	data=standard
EVENT	time=86	agent=robot_medium	type=container_delivered	data=container_9
EVENT	time=88	agent=robot_heavy_1	type=container_delivered	data=container_1
EVENT	time=92	agent=robot_heavy_2	type=container_delivered	data=container_4
EVENT	time=94	agent=robot_medium	type=container_delivered	data=container_5
EVENT	time=99	agent=robot_heavy_1	type=container_delivered	data=container_8
EVENT	time=103	agent=robot_heavy_2	type=container_delivered	data=container_10
EVENT	time=111	agent=robot_heavy_1	type=container_delivered	data=container_6
EVENT	time=111	agent=scheduler	type=deadline_ended	data=standard

Three of the four robots delivered something during this cycle (medium, heavy_1, heavy_2), which means the claim-based design is actually distributing work as intended. The cycle ended at T_0+29 s, which is way inside the $T_0+3\cdot\Delta T = T_0+90$ s limit, so no deadline_missed events appeared in this run. A second standard cycle ran from 134 s to 163 s with the same duration, so this is consistent and not a one-off.

9. Stages of Creation, Modification and Design

I worked on this project in three stages. Each one is described below, including what changed from the previous version and why.

9.1 Stage 1 — First Working Version

The first version had a much heavier environment. The Java code was the one deciding when storage was full, when a deadline was missed, and when to start or close a cycle. The .asl files for supervisor and scheduler were almost empty: they basically called Java actions and printed events on completion.

This version did print the right events on the console, but the BDI logic was on the wrong side — most of the reasoning was inside Java instead of inside the agents, which goes against the thin environment idea.

9.2 Stage 2 — Refactor to a Thin Environment

Two main changes happened here:

- All the decision logic moved from Java to the .asl files. `no_space_detected`, `deadline_missed`, cycle start and cycle close are agent decisions now.
- The environment publishes new state percepts: `shelf_state(Shelf, FreeWeight, FreeVolume)`, `output_pending(CId, Type)`, `stored_type(Type)`, `storage_busy(Type)`.

After this refactor the environment only updates physical state and publishes it as percepts. The supervisor now uses `storage_possible/4` over `shelf_state` to decide whether a container has any allowed shelf left, and the scheduler computes `T0` and the deadline by itself.

9.3 Stage 3 — Robustness and Edge Cases

After the refactor, several smaller things appeared during testing and were fixed one by one:

- Output cycle race condition: the `cycle_ready(Type)` flag (set 1500 ms after opening) protects the close logic from triggering too early.
- Shelf reservation: the environment now reserves capacity inside `claim_storage`, so two robots cannot fight over the same free slot.
- Robot stalls: BFS path planning treats other robots as obstacles, with retries and a `route_blocked` error after `MAX_REPATH_ATTEMPTS`. The robot then runs `recover_task`, which puts the container back in inbound.
- Repeated events: the `no_space_reported(Type)` belief makes sure each `no_space_detected` line is printed only once until the cycle closes.

What you see in the current code is the result of these three stages. All decisions are taken inside the .asl files, and the environment is genuinely thin.

10. Tests and Results

I tested the system with several scenarios on my local machine running `gradle run`. The right column reflects what I actually saw during the test run.

Test	Expected behavior	Result
Startup	GUI shows 4 robots and 9 shelves; no console errors.	OK

Test	Expected behavior	Result
Shelf policy	Urgent containers go only to S1/S5/S8; environment refuses other shelves.	OK
Standard output cycle	Supervisor and scheduler print the required events at T0; robots empty the shelves.	OK — captured at T0=82 and T0=134
Cycle close	deadline_ended printed only after the queue is empty.	OK — closed at 111 and 163
Single active cycle	While a cycle is active, no second cycle starts.	OK
Robot autonomy	All 4 robots claim tasks based on their own beliefs; medium and both heavy robots all appeared in deliveries during the same cycle.	OK
Deadline miss	If many containers are pending after the deadline, supervisor prints one event per container and the system continues.	Not triggered in this run — both standard cycles closed within ~29 s, well below the 90 s limit

Both observed cycles closed in 29 seconds (29 = 111–82 and also 163–134). The slowest single delivery in the cycle that started at 82 was container_6 with 29 seconds, while container_9 was done in just 4 seconds. Between cycles, new containers kept arriving and the system started again cleanly. To force a deadline_missed event, the inbound queue would have to be much longer than what the four robots can clear in 90 seconds; this did not happen in my test, but the supervisor logic for that case is in place (see section 6.4).

11. Difficulties Encountered

Three things took most of the development time:

- The thin environment refactor (stage 2). Moving decision logic out of Java and into .asl files, and figuring out which state percepts the supervisor and the scheduler actually needed.
- The output cycle race condition. The cycle was closing right after opening because the percepts had not arrived yet. Fixed with a 500 ms delay and the `cycle_ready(Type)` flag (see section 6.3).
- Robot conflicts on the grid. Two robots wanting the same cell or blocking each other. The BFS path planner treats other robots as obstacles, and `recover_task` releases the container if the route stays blocked too long.

There were also smaller issues — the `no_space_detected` event being printed every second, the shelf reservation race, the transport agent looping after the cycle closed — but those were fixed by adding the right beliefs or tightening the plan contexts.

12. Future Improvements

If I had more time, there are a few things I would try:

- A* path planning instead of BFS. It scales better than BFS and would reduce the number of recalculations when robots have to take long routes.
- Running parallel cycles for different types. Right now only one cycle is active at a time, but having one urgent cycle and one non-urgent cycle in parallel would shorten the wait when both areas fill up at the same time.
- Adaptive ΔT . Instead of a fixed 30 seconds, ΔT could be calculated from the average delivery time observed during the run.

13. Conclusion

In the end, the system does what was asked: shelves are used by container type, robots take their own decisions, output cycles open when a type runs out of space, urgent and non-urgent deadlines work with different time windows, time violations are reported but the system keeps running, and all the EVENT logs follow the required format.

What I really got out of this project was a much clearer understanding of how BDI works in practice — the difference between perceiving the world (beliefs from percepts), wanting something (goals like `!work_cycle`), and actually doing it through plans. The most useful lesson for me was probably that pushing decisions out of the environment and into the agents not only matches what the assignment wanted, but also makes the code easier to follow, because each agent ends up with one clear job.